

RAG (Retrieval Augmented Generation)



~~What is RAG?~~

Q Why we need RAG?

(Prompt)

Query

LLM

Response

LLM: Giant Transformers based neural networks

- We pretrain LLM on huge amount of data (Whole Internet)
- LLM stores their knowledge in parameters Data

How to access Knowledge

of parametric knowledge

as an User?

Ans

By Prompting

Parametric knowledge

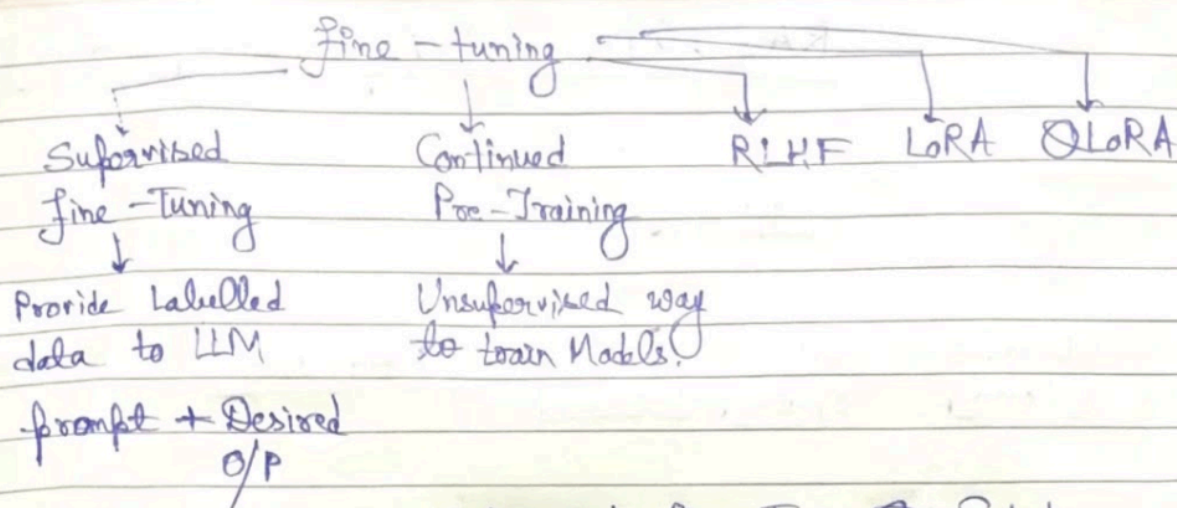
Problem: There are certain situations where this particular flow (Query → LLM → Response) doesn't help:

① Asking LLM about my private Data

② Asking Current Affairs / Latest fact from LLM
(Eg stock news 5 min ago) Recent Data

③ Hallucination → Gives sometimes factually incorrect info with confidence

Solⁿ: fine Tuning: you use pretrained LLM & retrain it on a smaller domain-specific dataset.



~~Supervised~~ Fine Tuning Steps

1. Collect Data: Prompts - Desired O/P

2. Choose a Method: ^{FT} QLoRA/LoRA, Full-parameter ~~FT~~ or Parameter efficient adapters

3. Train for few epochs: You keep the base wts frozen or partially frozen & Update only a small subset (LoRA) or all Weights (full FT)

4. Evaluate & Safety Test: Measure ^① exact-match, ^② factuality & ^③ hallucination rate against held-out data

1) You want LLM to work on Private Data. If we train LLM on Private data, prob is solved. (Private becomes part of Parameter Knowledge)

2) Recent data → fine tune should be done repeatedly & frequently

3) Hallucination → Add Tricky Prompts "I don't know" if you really do not know

Problems With Fine-Tuning :

- 1) Retraining model (LLM) $\rightarrow$ Computationally Expensive
 $\hookrightarrow$ Even if dataset is small, still model is huge
- 2) You need Strong technical expertise
- 3) Repetition of fine Tuning (When new info is coming at a very fast frequency)

Then Another Technique ?



In-Context Learning : Core Capability of LLM like GPT-3/4, Claude where the model can learn to solve a task purely by seeing examples in the prompt $\rightarrow$ Without updating its weight.

Prompt:

Eg 1 Below are Exs of text labelled with their sentiment. Use these Exs to determine the sentiment of final text.

Text: I love this phone. It's so smooth $\rightarrow$ Positive

Text: This app crashes a lot. $\rightarrow$ Negative

Text: The camera is amazing! $\rightarrow$ Positive

Text: I hate the battery life. $\rightarrow$

Eg 2 Prompt:

Label the named entities in the sentences (Person, Location, Organization)

Sentence: Elon Musk founded SpaceX

Entities: [Elon Musk $\rightarrow$ Person], [SpaceX $\rightarrow$ Organization]

Sentence: The Eiffel Tower is in Paris

Entities: [Eiffel Tower → ~~Location~~], [Paris → Location]

Sentence: Sundar Pichai Leads Google

Entities:

- In-Context Learning is an emergent property of LLM

Emergent Property → Behaviour or ability that suddenly appears in a system when it reaches a certain scale or complexity — even though it was not explicitly programmed or expected from the individual components.

[GPT-1
GPT-2]

→ These LLMs didn't show InContext learning.

→ If we give Ex in prompts to solve a task, these models were not able to learn new task from these Exs

[GPT-3]

→ Can learn from the Exs in prompt.

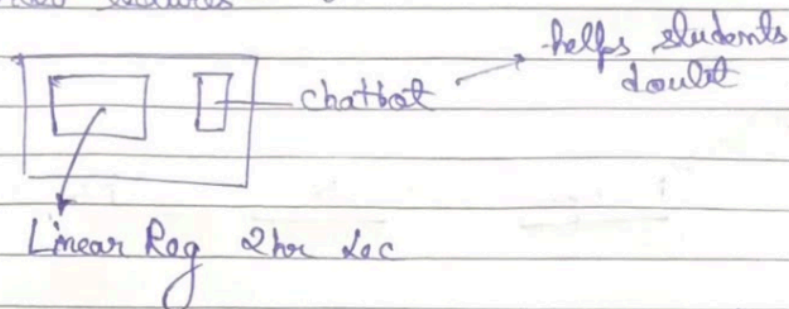
Paper
Published

→ [Large Models are few-shot learners.]

- Emergent property will not give you good results on every task.

- Instead of just example tasks, retrieve background info, facts, documents, product manuals etc & then inject that into the prompt to augment the model's knowledge.

Eg Website of Prityamshu where there are long video lectures



Spse student has a doubt in particular part (Gradient Descent) so he went & asked this doubt to LLM.

What we will do? Not only we will send student's question but also part of Lec where gradient descent is being taught

Let's say 5:00 to 25:00 min, We taught G.D. So transcript of this part is sent as context to the model.

Prompt:

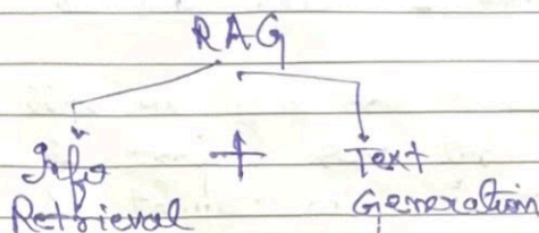
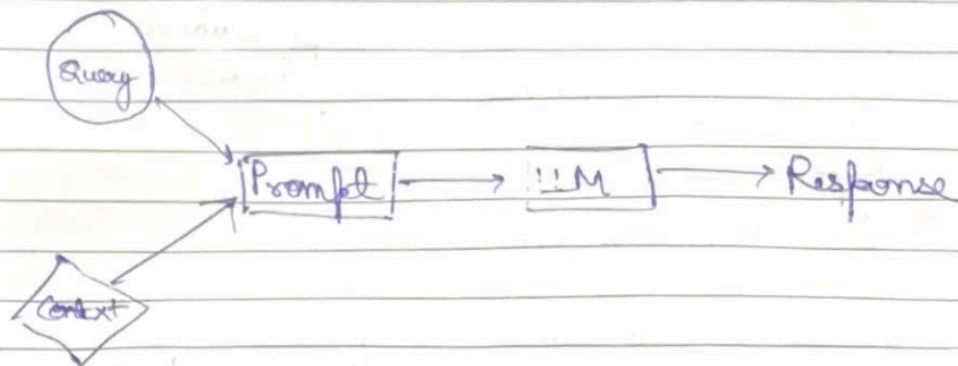
" " " You're a helpful assistant.

Answer the question ONLY from the provided context. If the context is insufficient, just say you don't know

{ context }

Question : { question } "

- RAG is a way to make LLM smarter by giving it extra information at the time you ask questions.

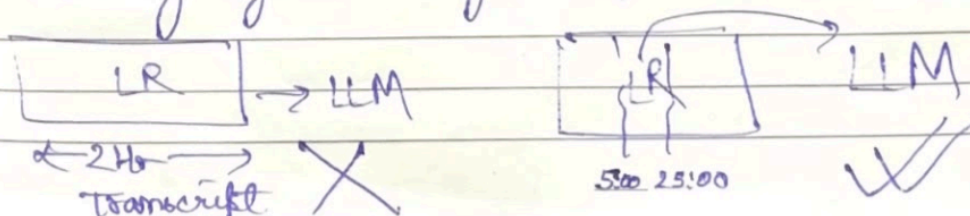


- RAG is divided into 4 steps:

- Indexing
- Retrieval
- Augmentation
- Generation

1) Indexing: Step in RAG when you create external knowledge base. Process of creating an external knowledge base is called Indexing.

2) Retrieval: You understand Query & go into ext knowledge base & try to find out which chunks will help in answering your Query & fetch those chunks.



3) Augmentation : Combining user's query & Retrieved context to create a prompt

→ Prompt Creation.

4) Generation : LLM uses its text-generation capabilities & in-context learning to answer this query & produces a response.

Indexing : Process of preparing your Knowledge base so that it can be efficiently searched at query time, This has 4 sub-steps :

a) Document Ingestion : You load your docs into memory.

Eg • PDFs, Word Docs, Youtube transcripts, Github Repos, SQL records, scraped webpages, blog pages etc.

Tools : Lanchain loader (PyPDFLoader, DirectoryLoader, YoutubeLoader, CSVLoader, WebBaseLoader, GitLoader, TextLoader, ~~etc~~, UnstructuredPDFLoader, PDFPlumberLoader, PyMuPDFLoader etc).

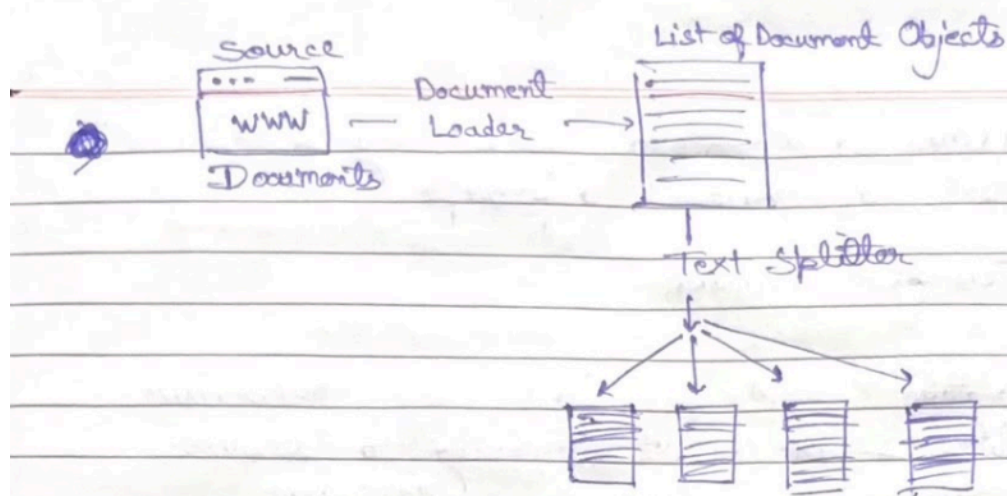
b) Text Chunking : Break large document into small, semantically meaningful chunks.

Q Why chunks?

→ LLMs have context limits (eg 4-32K Tokens)
→ Smaller chunks gives better embedding & Semantic Search

→ Semantic Based → Text Structure Based

Tools : SemanticChunker, RecursiveCharacterTextSplitter, MarkdownHeaderTextSplitter, RecursiveCharacterTextSplitter (Language = PYTHON), CharacterTextSplitter etc
→ Doc Structure Based → Length Based



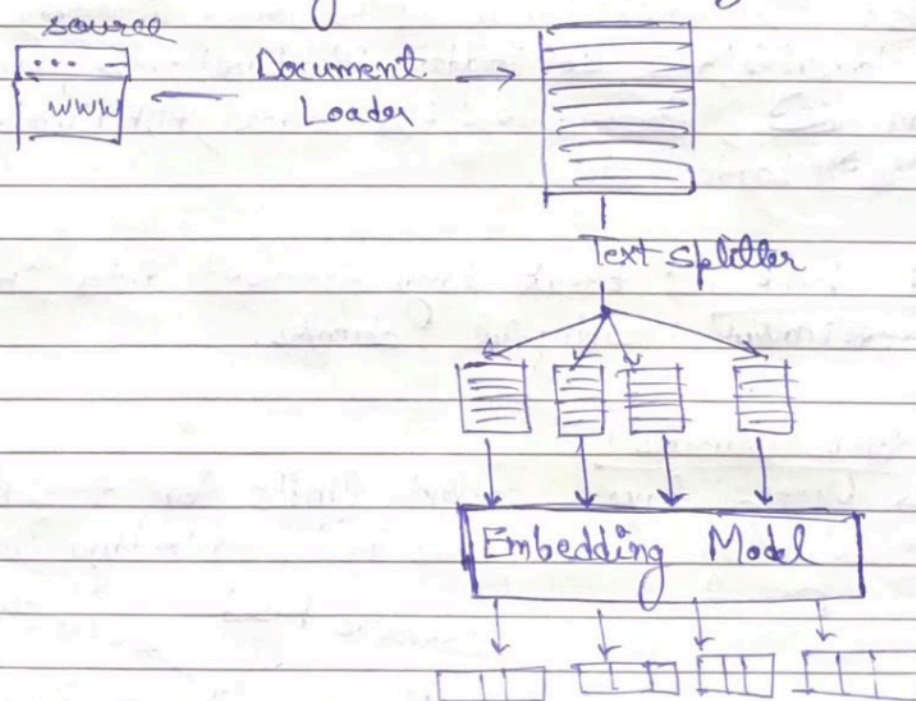
c) Embedding Generation: Convert each chunk into a dense vector (embedding) that captures its meaning.

Why Embeddings?

- Similar ideas close together in vector space
- Allows fast, fuzzy semantic search

Tools:

- OpenAI Embeddings
- HuggingFace Embeddings
- Instructor Embeddings
- Sentence Transformer Embeddings
- etc.

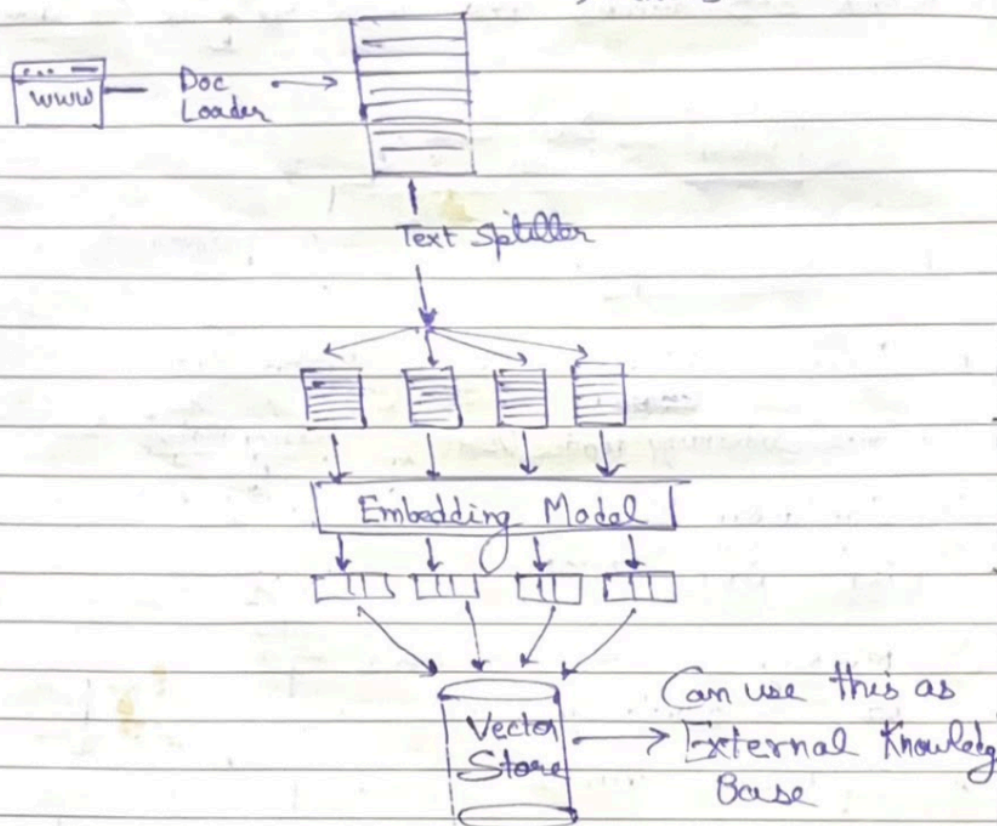


➤ Storage in Vector Store: Store the Vectors along with original chunk text + metadata in a vector database

Tools:

Vector DB Options:

- Local: FAISS, Chroma
- Cloud: Qdrant, Pinecone, Weaviate, Milvus



Retrieval: Retrieval is the real-time process of finding the most relevant pieces of info from a pre-built index (Created during indexing) based on User's Qs

Tools: On Basis of Source: WikipediaRetriever, ArxivRetriever

~~VectorStore~~ (or FAISS)
 VectorStore Retriever: Chroma → `vectorstore.as_retriever`
 (search_kwargs = {"k": 2})

On Basis of Type of Searching:

MMR → `vectorstore.as_retriever` (search_type = "mmr")

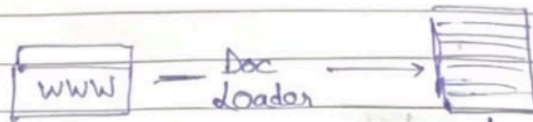
MultiQueryRetriever.from_llm(retriever, llm)

Contextual Compression Retriever (base-compressor, base-retriever)

Base Vector Store Retriever

for generate prompts
~~embedding~~
model

- Retrieval is like asking "From all the knowledge I have, which 3-5 chunks are most helpful to ans the query?"



Text Splitter

Retrieval Steps:

1) Query → Embedding Vector
Some Embedding model Used

2) Similarity Search
MAR MMR CCR

3) Ranking is performed on Docs
(Similarity Score)

4) Top^K Relevant chunks
are fetched

Query

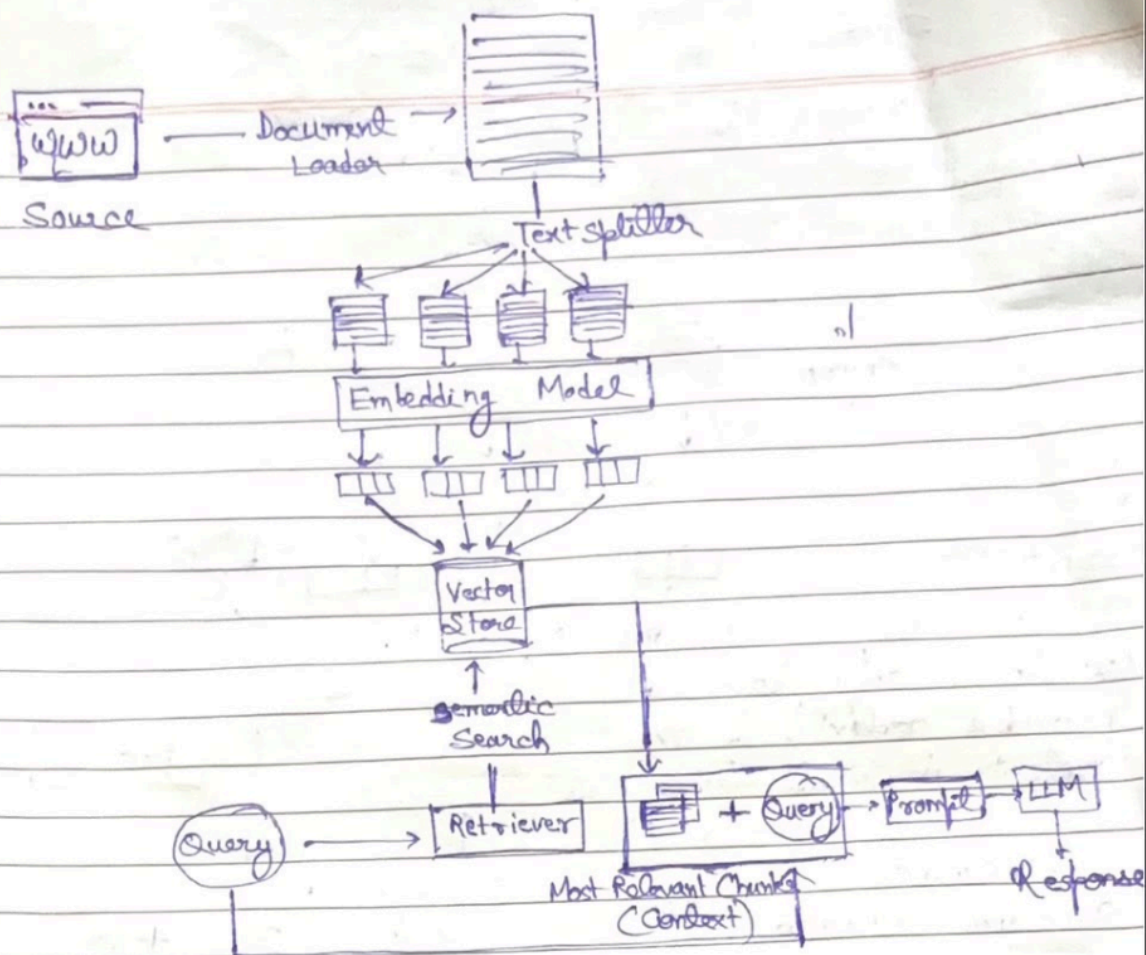
Semantic Search

Retriever

Most Relevant Chunks
(Context)

Augmentation

: Augmentation refers to the step where the retrieved documents (chunks of relevant context) are combined with user's query to form a new, enriched prompt for LLM.



Problems:

- 1> Private Data
- 2> Recent Developments
- 3> Hallucination

1> RAG external knowledge base is made of private data so LLM has to access to private data

2> Recent Articles, Recent news, info → add these into your knowledge base
(No need of repeated fine Tuning)
- Less Costlier

3> Providing exact context & telling the model to ans the question from provided context only. If the context is insufficient, just say You don't know